

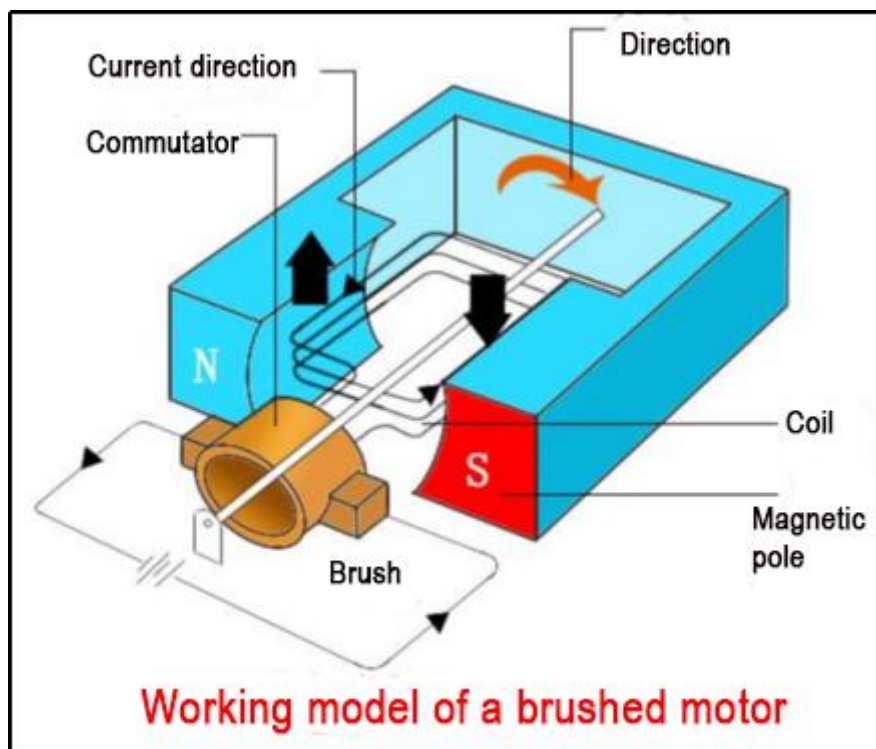
Lesson 9 RRCLite Brushed DC Motor Program Analysis

1. Preface

The DC motors available in the market are mainly divided into two types: brushless motors and brushed motors. This chapter focuses on explaining brushed DC motors.

Brushed Motors:

In brushed DC motors, current flows through the armature coil, generating a magnetic field that interacts with the magnetic field of the permanent magnet, causing the motor to rotate. The following diagram illustrates the working model of a brushed DC motor.



Components: Permanent magnet, armature coil, brushes, bearings. Speed control principle: By changing the voltage of the power supply, the force on the rotor coil will also change, thereby changing the motor speed. Commutation principle: Changing the direction of current in the coil can achieve motor commutation.

Brushless Motors:

Working principle: The stator of a brushless DC motor is a coil winding armature, and the rotor is a permanent magnet. The real-time position of the motor rotor is detected, and then different phase currents are applied to the motor according to the rotor position, so that the stator generates a uniformly changing rotational magnetic field, allowing the motor to follow the magnetic field and rotate.

In the previous section, we learned about the principle of brushed DC motors. In this chapter, we will learn about the source code of brushed DC motors. To do this, we need to prepare STM32CubeIDE or Keil5, and a brushed DC motor.

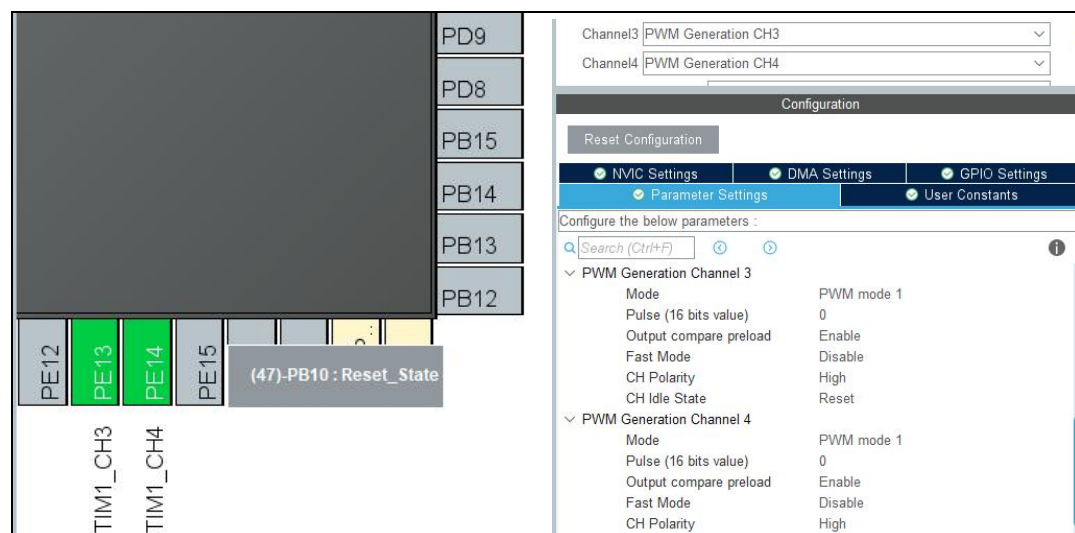
2. Program Storage Path

The example routine for this particular content is named "Motor_Demo":

Program path: Motor_Demo\motor_porting.c

STM32CubeMX path: Motor_Demo\Motor_Demo.ioc

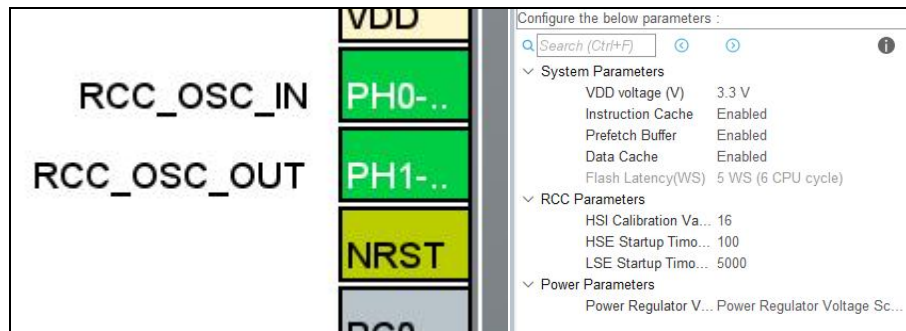
3. Configure the Timer



In STM32CubeIDE, open the STM32CubeMX Configurator. In the Peripherals section, locate TIM1 and enable it. Then, in the Configuration tab, set TIM1 for PWM Generation CH3 and CH4. This will be used for generating PWM signals.

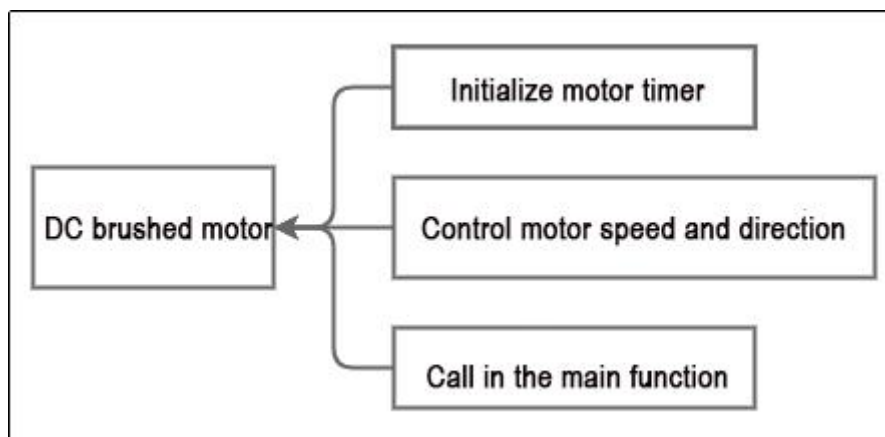
Next, in the STM32CubeMX Configurator, go to TIM1 Configuration. In the PWM Generation Channel 3 and PWM Generation Channel 4 sections, set the appropriate PWM frequency and duty cycle. Click OK to save the settings. Click on Project > Generate Code to generate the initialization code.

4. Clock configuration



5. Program Analysis

Program Structure Diagram



5.1 Initialize Motor Timer

```
motor_init();
```

```
void motors_init(void)
{
    for(int i = 0; i < 4; ++i) {
        motors[i] = LWMEM_CCM_MALLOC(sizeof( EncoderMotorObjectTypeDef));
        encoder_motor_object_init(motors[i]);
        motors[i]->ticks_overflow = 60000;
        motors[i]->ticks_per_circle = MOTOR_DEFAULT_TICKS_PER_CIRCLE;
        motors[i]->rps_limit = MOTOR_DEFAULT_RPS_LIMIT;
        motors[i]->pid_controller.set_point = 0.0f;
        motors[i]->pid_controller.kp = MOTOR_DEFAULT_PID_KP;
        motors[i]->pid_controller.ki = MOTOR_DEFAULT_PID_KI;
        motors[i]->pid_controller.kd = MOTOR_DEFAULT_PID_KD;
    }

    /* Motor 1 */
    motors[0]->set_pulse = motor1_set_pulse;
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    __HAL_TIM_ENABLE(&htim1);
    __HAL_TIM_MOE_ENABLE(&htim1);

    /* encoder */
    __HAL_TIM_SET_COUNTER(&htim5, 0);
    __HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE(&htim5);
    HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL);
}
```

Note: Every time the motor is used, please make sure that the speed limit, PID, and other parameters used in the motor initialization function match the motor model currently in use.

The motor model can be checked in `motors_param.h`.

Reset the counter of the TIM1 timer using the `__HAL_TIM_SET_COUNTER()` function.

Start the TIM1 timer counting using the `__HAL_TIM_ENABLE()` function.

Enable the Main Output Enable (MOE) of the TIM1 timer to enable PWM signal output using the `__HAL_TIM_MOE_ENABLE()` function.

5.2 Control Motor Speed and Direction

```
static void motor1_set_pulse(EncoderMotorObjectTypeDef *self, int speed)
{
    if(speed > 0) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, speed);
    } else if(speed < 0) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, -speed);
    } else {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, 0);
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, 0);
    }
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
    HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);
}
```

The function `motor1_set_pulse()` controls a DC motor by adjusting the duty cycle of the PWM signal, thereby controlling the motor's rotation speed and direction.

If the value of 'speed' is greater than 0, the motor will rotate forward. The function sets the duty cycle of `TIM_CHANNEL_3` to 0 and sets the duty cycle

of TIM_CHANNEL_4 to 'speed'.

If the value of 'speed' is less than 0, the motor will rotate backward. The function sets the duty cycle of TIM_CHANNEL_4 to 0 and sets the duty cycle of TIM_CHANNEL_3 to the absolute value of 'speed'.

If the value of 'speed' is equal to 0, the motor will stop. The function sets the duty cycle of both TIM_CHANNEL_3 and TIM_CHANNEL_4 to 0.

Finally, the function starts the generation of PWM signals on TIM_CHANNEL_3 and TIM_CHANNEL_4.

5.3 Call in the app_task_entry Function

```
void app_task_entry(void *argument)
{
    extern osTimerId_t led_timerHandle;
    extern osTimerId_t buzzer_timerHandle;
    extern osTimerId_t button_timerHandle;
    extern osTimerId_t battery_check_timerHandle;
    extern osMessageQueueId_t moving_ctrl_queueHandle;

    leds_init();
    motors_init();
    pwm_servos_init();
    serial_servo_init();
    buzzers_init();
    buttons_init();
    WS2812b_Configuration();

    button_register_callback(buttons[0], button_event_callback);
    button_register_callback(buttons[1], button_event_callback);

    osTimerStart(led_timerHandle, LED_TASK_PERIOD);
    osTimerStart(buzzer_timerHandle, BUZZER_TASK_PERIOD);
    osTimerStart(button_timerHandle, BUTTON_TASK_PERIOD);
    osTimerStart(battery_check_timerHandle, BATTERY_TASK_PERIOD);
    packet_handle_init();
}
```

At the beginning of the task, call the function `motors_init()` to initialize the motor structure used for controlling the motor.

```
for(;;) {
    motor1_set_pulse(motors[0], 500);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], 1000);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], 500);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], 0);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], -500);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], -1000);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], -500);
    HAL_Delay(2000);
    motor1_set_pulse(motors[0], 0);
    HAL_Delay(2000);
}
```

In the for loop, call the motor1_set_pulse function on the motors[0] motor structure object to change the current PWM duty cycle variable of counter 1. This changes the motor speed.

```
/**
 * @brief Encoder motor object structure
 */
struct EncoderMotorObject{
    int64_t counter;          /**< @brief Total count value, assumes no overflow with 64-bit integer */
    int64_t overflow_num;     /**< @brief Overflow count value */
    int32_t ticks_overflow;   /**< @brief Count overflow value */
    float tps;                /**< @brief the frequency of the counter */
    float rps;                /**< @brief the output shaft speed in units of revolutions per second */
    int current_pulse;        /**< @brief The current PWM value of the output, with the sign indicating the direction of rotation */
    PID_ControllerTypeDef pid_controller; /**< @brief PID controller */

    /** Porting portable hardware interface */
    int32_t ticks_per_circle; /**< @brief The number of counts generated by one rotation of the motor output shaft, according to the
float rps_limit; /**< @brief The maximum speed of the motor, which is usually set to a slightly lower value than the speed at 100
    /**
     * @brief set motor speed -1000-1000
     * @param encoder motor object
     * @param the new PWM value, ranging from -1000-1000, corresponding to 0-100% speed in both forward and reverse directions
     * @retval None.
     */
    void (*set_pulse)(EncoderMotorObjectTypeDef *self, int pulse);
};
```

The above diagram shows the structure of the encoder motor object's structure. For more details, please refer to the comments.